

SQL - Funkcje okna (Window functions)

Lab 2

Imiona i nazwiska: Katarzyna Wesołowska, Paweł Moćko - POPRAWIONY RAPORT Z LAB2

Celem ćwiczenia jest zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu "tradycyjnych" konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

-- ...

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022
- PostgreSQL - wersja 15/16/17
- SQLite
- Narzędzia do komunikacji z bazą danych
 - SSMS - Microsoft SQL Managment Studio
 - DtataGrip lub DBeaver
- Przykładowa baza Northwind/Northwind3
 - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apres 2019
- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020

- Kilka linków do materiałów które mogą być pomocne - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16>
 - <https://www.sqlservertutorial.net/sql-server-window-functions/>
 - <https://www.sqlshack.com/use-window-functions-sql-server/>
 - <https://www.postgresql.org/docs/current/tutorial-window.html>
 - <https://www.postgresqltutorial.com/postgresql-window-function/>
 - <https://www.sqlite.org/windowfunctions.html>
 - <https://www.sqlitetutorial.net/sqlite-window-functions/>
- W razie potrzeby - opis ikonek używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
 - <https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical/operators-reference>

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
 - na lokalnym serwerze MS SQL
 - na lokalnym serwerze PostgreSQL
 - z lokalną bazą SQLite

Można też skorzystać z innych narzędzi klienckich (wg własnego uznania)

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Korzystamy ze "zmodyfikowanej wersji" bazy northwind

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Zadanie 1

Funkcje rankingu, `row_number()`, `rank()`, `dense_rank()`

```
select productid, productname, unitprice, categoryid,  
       row_number() over(partition by categoryid order by unitprice desc) as rowno,  
       rank() over(partition by categoryid order by unitprice desc) as rankprice,  
       dense_rank() over(partition by categoryid order by unitprice desc) as  
denserankprice  
from products;
```

Wykonaj polecenie, zaobserwuj wynik. Porównaj funkcje row_number(), rank(), dense_rank(). Skomentuj wyniki.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

Obserwacje:

- zapytanie numeruje produkty osobno w każdej kategorii (partition by categoryid), ustawiając je od najdroższego do najtańszego (order by unitprice desc).
- row_number -> nadaje każdemu wierszowi unikalny numer w danej kategorii. Nawet jeśli dwa produkty mają taką samą cenę, dostaną różne numery, np. 1 i 2.
- dla rank() -> taka sama ranga jest nadawana dla tej samej wartości (tu ceny), a następnie następuje przeskok do następnej liczby, która faktycznie powinna pojawić się w tej kolejności. Czyli; np: jeśli dwa produkty mają tę samą cenę i zajmują pierwsze miejsce, oba dostaną rangę 1, a kolejny produkt dostanie rangę 3.
- dla dense_rank() -> działa podobnie jak rank(), ale nie pozostawia luk. Jeśli dwa produkty mają tę samą cenę i oba dostaną 1, to kolejny produkt dostanie 2.

Wybrany system: PostgreSQL

Funkcje rankingu bez użycia funkcji okna

```
select
  p1.productid,
  p1.productname,
  p1.unitprice,
  p1.categoryid,

  (
    select count(*)
    from products p2
    where p2.categoryid = p1.categoryid
      and (
        p2.unitprice > p1.unitprice
        or (p2.unitprice = p1.unitprice and p2.productid <= p1.productid)
      )
  ) as row_number_calc,

  (
    select count(*)
    from products p2
    where p2.categoryid = p1.categoryid
      and p2.unitprice > p1.unitprice
  ) + 1 as rank_calc,

  (
    select 1 + count(distinct p2.unitprice)
    from products p2
```

```
        where p2.categoryid = p1.categoryid
              and p2.unitprice > p1.unitprice
      ) as dense_rank_calc

from products p1
order by p1.categoryid, p1.unitprice desc, p1.productid;
```

Komentarz:

- row_number_calc: zlicza wszystkie produkty droższe oraz te z tą samą ceną i mniejszym productid, dzięki czemu każdy wiersz dostaje unikalny numer.
- rank_calc: zlicza tylko produkty droższe i dodaje 1, więc produkty z tą samą ceną mają tę samą rangę, ale pojawiają się luki.
- dense_rank_calc: bierze pod uwagę tylko unikatowe (distinct) wartości. Liczy ceny większe od bieżącej i dodaje 1, więc nie ma luk w numeracji.
- W wersji bez funkcji okna zapytanie jest zdecydowanie dłuższe i mniej czytelne. Udało się uzyskać podobny wynik. Różnica jest taka, że jest inna kolejność dla remisów dla części "row_number_calc" - została ona ustalona przez productid. W funkcji okna z polecenia kolejność nie została określona.

Zadanie 2

Baza: Northwind, tabela product_history

Dla każdego produktu, podaj 4 najwyższe ceny tego produktu w danym roku. Zbiór wynikowy powinien zawierać:

- rok
- id produktu
- nazwę produktu
- cenę
- datę (datę uzyskania przez produkt takiej ceny)
- pozycję w rankingu
- Uporządkuj wynik wg roku, nr produktu, pozycji w rankingu

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

Dla PostgreSQL:

```
EXPLAIN (ANALYZE, BUFFERS)
WITH ranked_prices AS (
    SELECT
        EXTRACT(YEAR FROM ph.date) AS rok,
        ph.productid,
        ph.productname,
        ph.unitprice AS cena,
        ph.date AS data_ceny,
        ROW_NUMBER() OVER (
            PARTITION BY EXTRACT(YEAR FROM ph.date), ph.productid
            ORDER BY ph.unitprice DESC, ph.date DESC, ph.id DESC
        ) AS pozycja
    FROM product_history ph
    WHERE ph.date IS NOT NULL
        AND ph.unitprice IS NOT NULL
        AND ph.productid IS NOT NULL
)
SELECT
    rok,
    productid,
    productname,
    cena,
    data_ceny,
    pozycja
FROM ranked_prices
WHERE pozycja <= 4
ORDER BY rok, productid, pozycja;
```

Bez funkcji okna

```
WITH base AS (
    SELECT
        ph.id,
        EXTRACT(YEAR FROM ph.date)::int AS rok,
        ph.productid,
        ph.productname,
        ph.unitprice AS cena,
        ph.date AS data_ceny
    FROM product_history ph
    WHERE ph.date IS NOT NULL
        AND ph.unitprice IS NOT NULL
        AND ph.productid IS NOT NULL
)
SELECT
    b.rok,
    b.productid,
    b.productname,
    b.cena,
    b.data_ceny,
    COUNT(b2.id) + 1 AS pozycja
```

```

FROM base b
LEFT JOIN base b2
  ON b2.rok = b.rok
  AND b2.productid = b.productid
  AND (
    b2.cena > b.cena
    OR (b2.cena = b.cena AND b2.data_ceny > b.data_ceny)
    OR (b2.cena = b.cena AND b2.data_ceny = b.data_ceny AND b2.id > b.id)
  )
GROUP BY
  b.id, b.rok, b.productid, b.productname, b.cena, b.data_ceny
HAVING COUNT(b2.id) < 4
ORDER BY
  b.rok, b.productid, pozycja;

```

SZBD	Wariant zapytania	Liczba zwróconych wierszy	Czas wykonania	Koszt zapytania
PostgreSQL	z funkcją okna	25564	5 678.781 ms	685985.00
PostgreSQL	bez funkcji okna	25564	14 000 ms	23363656.52
MS SQL Server	z funkcją okna	25564	1 428 ms	50.6744
MS SQL Server	bez funkcji okna	25564	1 min 40 s	152.551
SQLite	z funkcją okna	25564	13 000ms	NA
SQLite	bez funkcji okna	-	18min+ (przerwano)	NA

Wnioski:

- Dla metody z funkcją okna najszybszy był MS SQL Server, potem PostgreSQL, a najwolniejszy SQLite.
- Wersja bez funkcji okna działa przez porównanie każdego rekordu z innymi rekordami tego samego produktu i roku. Następnie zliczenie, ile wierszy ma wyższą cenę lub (jeśli remis) późniejszą datę albo większe id. Pozwala to wyznaczyć pozycję w rankingu.
- Wariant z funkcją okna był wyraźnie wydajniejszy we wszystkich testowanych SZBD: miał krótszy czas wykonania i niższy koszt niż wersja bez funkcji okna.
- Wersja bez funkcji okna w SQLite zapytanie było tak wolne, że test przerwano.

Zadanie 3

Funkcje `lag()`, `lead()`

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `lag()`, `lead()`

```
select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid order by date)
as previousprodprice,
       lead(unitprice) over (partition by productid order by date)
as nextprodprice
from product_history
where productid = 1 and year(date) = 2022
order by date;

with t as (select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid
order by date) as previousprodprice,
       lead(unitprice) over (partition by productid
order by date) as nextprodprice
       from product_history
       )
select * from t
where productid = 1 and year(date) = 2022
order by date;
```

Jak działają funkcje `lag()`, `lead()`?

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

Funkcja okna z polecenia w PostgreSQL:

```
select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid order by date)
as previousprodprice,
       lead(unitprice) over (partition by productid order by date)
as nextprodprice
from product_history
where productid = 1 and EXTRACT(YEAR FROM date) = 2022
order by date;

with t as (select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid
order by date) as previousprodprice,
       lead(unitprice) over (partition by productid
order by date) as nextprodprice
       from product_history
       )
select * from t
where productid = 1 and EXTRACT(YEAR FROM date) = 2022
order by date;
```

Obserwacje:

- lag(unitprice) zwraca cenę z poprzedniego wiersza dla produktu
- lead(unitprice) zwraca cenę z następnego wiersza
- Dla pierwszego wiersza LAG() zwraca NULL, a dla ostatniego LEAD() zwraca NULL.
- W pierwszym zapytaniu funkcje lag() i lead() działają tylko na rekordach z 2022 roku dla produktu 1, ponieważ filtrowanie następuje przed obliczeniem funkcji okna.
- W drugim zapytaniu lag() i lead() są liczone na całej tabeli, a dopiero potem wyniki są filtrowane, dlatego poprzednia lub następna wartość może pochodzić spoza 2022 roku.

Bez funkcji okna (w PostgreSQL):

```
WITH base AS (  
    SELECT  
        id,  
        productid,  
        productname,  
        categoryid,  
        date,  
        unitprice  
    FROM product_history  
    WHERE productid = 1  
        AND EXTRACT(YEAR FROM date) = 2022  
)  
SELECT  
    b.productid,  
    b.productname,  
    b.categoryid,  
    b.date,  
    b.unitprice,  
    p.unitprice AS previousprodprice,  
    n.unitprice AS nextprodprice  
FROM base b  
LEFT JOIN LATERAL (  
    SELECT ph_prev.unitprice  
    FROM product_history ph_prev  
    WHERE ph_prev.productid = b.productid  
        AND (  
            ph_prev.date < b.date  
            OR (ph_prev.date = b.date AND ph_prev.id < b.id)  
        )  
    ORDER BY ph_prev.date DESC, ph_prev.id DESC  
    LIMIT 1  
) p ON true  
LEFT JOIN LATERAL (  
    SELECT ph_next.unitprice  
    FROM product_history ph_next  
    WHERE ph_next.productid = b.productid  
        AND (  
            ph_next.date > b.date  
            OR (ph_next.date = b.date AND ph_next.id > b.id)  
        )  
)
```



```
ORDER BY ph_next.date ASC, ph_next.id ASC
LIMIT 1
) n ON true
ORDER BY b.date, b.id;
```

Komentarz:

- W zapytaniu bez funkcji okna użyto JOIN LATERAL, który jest składnią typową dla PostgreSQL. W MS SQL podobną rolę pełni CROSS i OUTER APPLY, gdzie prawa strona jest obliczana dla każdego wiersza lewej strony. W SQLite odpowiednikiem są podzapytania skorelowane.
- LATERAL pozwala podzapytaniu w FROM odwoływać się do kolumn z wiersza aktualnie przetwarzanego w zapytaniu głównym. Wykorzystano to tutaj, aby osobno wyznaczyć poprzedni i następny wiersz, co w tym zadaniu zastępuje działanie funkcji lag() i lead().
- Zapytanie odpowiada drugiemu wariantowi z polecenia, czyli temu, w którym lag() i lead() są wyznaczone najpierw dla całej tabeli, a dopiero potem wyniki są filtrowane do produktu 1 i roku 2022. Oznacza to, że poprzednia i następna cena mogą pochodzić także spoza 2022 roku.
- Zapytanie z oknem jest o wiele szybsze: czas wykonania poniżej 0,4s, gdzie wersja z LATERAL to prawie 40s czasu wykonania. To samo z kosztem: 48134.18 vs 54029.88.

Zadanie 4

Baza: Northwind, tabele customers, orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- nazwę klienta, nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- nr poprzedniego zamówienia danego klienta,
- datę poprzedniego zamówienia danego klienta,
- wartość poprzedniego zamówienia danego klienta.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

```
SELECT
  c.companyname AS customername,
  o.orderid,
  o.orderdate,
  COALESCE(odsum.ordervalue, 0) + COALESCE(o.freight, 0) AS ordervalue,
  LAG(o.orderid) OVER w AS previousorderid,
  LAG(o.orderdate) OVER w AS previousorderdate,
  LAG(COALESCE(odsum.ordervalue, 0) + COALESCE(o.freight, 0)) OVER w AS
previousordervalue
```

```
FROM orders o
JOIN customers c
  ON c.customerid = o.customerid
LEFT JOIN (
  SELECT
    od.orderid,
    SUM(od.unitprice * od.quantity * (1 - od.discount)) AS ordervalue
  FROM orderdetails od
  GROUP BY od.orderid
) odsum
  ON odsum.orderid = o.orderid
WINDOW w AS (
  PARTITION BY o.customerid
  ORDER BY o.orderdate, o.orderid
)
ORDER BY c.companyname, o.orderdate, o.orderid;
```

Komentarz:

- Zapytanie najpierw oblicza wartość każdego zamówienia na podstawie tabeli orderdetails, sumując wartości wszystkich pozycji: $\text{unitprice} * \text{quantity} * (1 - \text{discount})$. Do tak obliczonej kwoty dodawany jest potem koszt przesyłki z tabeli orders.
- Funkcja okna LAG() została użyta do pobrania informacji o poprzednim zamówieniu tego samego klienta. Dzięki PARTITION BY o.customerid historia zamówień jest liczona osobno dla każdego klienta.
- WINDOW w AS (...) upraszcza zapis, ponieważ pozwala nadać wspólną nazwę ustawieniom okna. Dzięki temu nie trzeba kilka razy powtarzać tego samego PARTITION BY i ORDER BY, więc zapytanie jest krótsze i czytelniejsze.
- COALESCE(..., 0) -> brak kosztu przesyłki albo brak wyliczonej sumy nie powoduje, że cała wartość zamówienia staje się pusta.
- Dla pierwszego zamówienia danego klienta funkcja LAG() zwraca NULL, ponieważ nie istnieje wcześniejsze zamówienie, do którego można się odwołać.

Zadanie 5

Funkcje `first_value()`, `last_value()`

Baza: Northwind, tabele customers, orders, order details

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `first_value()`, `last_value()`.

Skomentuj uzyskane wyniki. Czy funkcja `first_value` pokazuje w tym przypadku najdroższy produkt w danej kategorii, czy funkcja `last_value()` pokazuje najtańszy produkt?

Co jest przyczyną takiego działania funkcji `last_value`.

Co trzeba zmienić żeby funkcja `last_value` pokazywała najtańszy produkt w danej kategorii?

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

```
select productid, productname, unitprice, categoryid,  
       first_value(productname) over (partition by categoryid  
order by unitprice desc) first,  
       last_value(productname) over (partition by categoryid  
order by unitprice desc) last  
from products  
order by categoryid, unitprice desc;
```

Wyniki:

Wykonaliśmy zapytanie

```
select productid,  
       productname,  
       unitprice,  
       categoryid,  
       first_value(productname) over w first,  
       last_value(productname) over w last  
from products  
window w as (partition by categoryid  
              order by unitprice desc)  
order by categoryid, unitprice desc;
```

Dla danego wiersza, zwrócony zostaje pierwszy bądź ostatni wiersz z zakresu działania funkcji okna (*window_frame*).

Dla powyższego zapytania otrzymujemy pierwszą oraz ostatnią wartość w takim zakresie.

► Wyniki dla `categoryId=7`

Tutaj warto zaznaczyć, które wiersze zawarte są we wcześniej wspomnianym zakresie. Przy użyciu sortowania, funkcje okna działają w zakresie od pierwszego wiersza tabeli (bądź jej partycji), do danego wiersza.

Dla lepszego zobrazowania tego mechanizmu przygotowaliśmy poniższe zapytanie (działające wyłącznie dla PostgreSQL)

```
select  
  productname,  
  unitprice,  
  array_agg(productname) over w as frame_contents  
from products  
where categoryId = 7  
window w as (  
  partition by categoryId  
  order by unitprice desc
```

```
)  
order by unitprice desc;
```

► Wyniki

Obrazuje ono, że dla każdego kolejnego wiersza, ilość rozpatrywanych elementów się zwiększa.

Co trzeba zmienić żeby funkcja `last_value` pokazywała najtańszy produkt w danej kategorii?

Należy zmienić sposób sortowania wg `unitprice`, używając kolejności rosnącej zamiast malejącej.

Zadanie 6

Baza: Northwind, tabele orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- Id klienta,
- nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- dane zamówienia klienta o najniższej wartości w danym miesiącu
 - nr zamówienia o najniższej wartości w danym miesiącu
 - datę tego zamówienia
 - wartość tego zamówienia
- dane zamówienia klienta o najwyższej wartości w danym miesiącu
 - nr zamówienia o najniższej wartości w danym miesiącu
 - datę tego zamówienia
 - wartość tego zamówienia

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

```
select customerId,  
       orderId,  
       orderDate,  
       value,  
       first_value(orderid) over w  as min_value_order,  
       first_value(orderdate) over w as min_value_date,  
       first_value(value) over w    as min_value  
from base
```

```
window w as (  
    partition by extract(month from orderdate)  
    order by value  
)
```

Powyższe zapytanie zadziała dla PostgreSQL, w celu wykonania analogicznego dla MS SQL lub SQLite należy zmodyfikować kod analogicznie jak w zadaniu 2. Dla SQLite:

```
@@ -14,6 +14,6 @@  
    first_value(value) over w      as min_value  
from base  
window w as (  
-    partition by extract(month from orderdate)  
+    partition by strftime('%M', orderdate)  
    order by value  
)
```

Dla MS SQL:

```
@@ -14,6 +14,6 @@  
    first_value(value) over w      as min_value  
from base  
window w as (  
-    partition by extract(month from orderdate)  
+    partition by month(orderdate)  
    order by value  
)
```

Zadanie 7

Baza: Northwind, tabela product_history

Napisz polecenie które pokaże wartość sprzedaży każdego produktu narastająco od początku każdego miesiąca. Użyj funkcji okna

Zbiór wynikowy powinien zawierać:

- id pozycji
- id produktu
- datę
- wartość sprzedaży produktu w danym dniu
- wartość sprzedaży produktu narastające od początku miesiąca

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

funkcje okna

```
select id,
       productid,
       date,
       value,
       sum(value) over curr_month as value_so_far
from product_history
window
  curr_month as (
    partition by productid, extract(month from date)
    order by date
  )
order by productid, date
```

Powyższe zapytanie działa dla PostgreSQL, dla MS SQL oraz SQLite można dokonać analogicznych zmian, jak w poprzednim zadaniu.

Bez funkcji okna

```
select ph1.id,
       ph1.productid,
       ph1.date,
       ph1.value,
       (
         select sum(ph2.value)
         from product_history ph2
         where ph1.productid = ph2.productid
               and extract(month from ph1.date) = extract(month from ph2.date)
               and ph2.date <= ph1.date
       ) as value_so_far
from product_history ph1
order by ph1.productid, ph1.date
```

Zdecydowaliśmy się użyć podzapytania, które sumuje wszystkie wiersze dla danego produktu i wiersza, które występują wcześniej (chronologicznie) niż dany wiersz.

Czasy wykonania poszczególnych zapytań różniły się znacznie od serwera bazy danych.

Serwer bazy danych	Czas wykonania użyciu funkcji okna [s]	Czas wykonania przy użyciu podzapytania
PostgreSQL	2.765	2.265

Serwer bazy danych	Czas wykonania użyciu funkcji okna [s]	Czas wykonania przy użyciu podzapytania
SQLite	10.515	> 60
MS SQL	2.395	> 60

Czas wykonywania zapytania był znacznie zredukowany dla SQLite oraz MS SQL. Dla PostgreSQL był on zbliżony (z przewagą zapytania z podzapytaniem). Jest to prawdopodobnie efekt optymalizacji podzapytań.

Zadanie 8

Wykonaj kilka "własnych" przykładowych analiz.

Czy są jeszcze jakieś ciekawe/przydatne funkcje okna (z których nie korzystałeś w ćwiczeniu)? Spróbuj ich użyć w zaprezentowanych przykładach.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

Średnia ruchoma

Możemy modyfikować zakres działania okna, aby rozważać zadaną liczbę wierszy przed oraz po danym wierszu.

```
```sql
SELECT
 id,
 productid,
 date,
 value,
 AVG(value) OVER moving_7d as avg_7d
FROM product_history
window
 moving_7d as (
 partition by productid
 order by date
 rows between 6 preceding and current row
)
ORDER BY productid, date
```

### RANGE a ROWS

Powyższy przykład tworzy średnią ruchomą dla siedmiu dni, przy użyciu słowa kluczowego `rows`. Jest to błąd, ponieważ wówczas okno jest tworzone dla określonej liczby wierszy, niezważwszy na różnicę wartości między wierszami. W przypadku, gdy taka różnica jest potrzebna - należy wykorzystać `range`.

```
SELECT
 productid,
 productname,
 date,
 unitprice,
 value,
 AVG(value) OVER true_7d AS avg_value_7d_range,
 AVG(value) OVER fake_7d AS avg_value_7d_rows
FROM product_history
window
 -- RANGE: zawsze rozważamy wartości oddaloną o daną ilość
 -- od wiersza. Tutaj: zawsze zakres 7 dni
 true_7d as (
 partition by productid
 order by date
 range between interval '6 days' preceding and current row
),
 -- ROWS: zawsze wybiera 6 poprzednich dat,
 -- nawet w przypadku, gdy dla pewnego dnia nie ma wartości.
 fake_7d as (
 partition by productid
 order by date
 rows between 6 preceding and current row
)
ORDER BY productid, date
```

## Ranking procentowy

Oprócz `rank` oraz `dense_rank` istnieje również funkcja `percent_rank`, która przyporządkowuje procentową ilość poprzedzających wierszy.

Wykorzystując ją możemy np. zdobyć top 25% produktów w swojej kategorii, wg wartości, dla każdego miesiąca

```
with ranked as (select productid,
 productname,
 categoryid,
 date,
 value,
 PERCENT_RANK() over monthly_categories as pct_rank
from product_history
window monthly_categories as (
 partition by categoryid, extract(month from date)
 order by value
)
order by categoryid, date, pct_rank
)
select *
from ranked
```



```
where pct_rank > 0.75
order by categoryid, date, pct_rank
```

---

## Punktacja

zadanie	pkt
1	1
2	2
3	1
4	1
5	1
6	1
7	2
8	2
razem	11